

bcinr-powl v2: The Chatman Equation and the Picosecond Enterprise

$$\mathcal{R} \vdash \mathcal{A} = \mu(\mathcal{O}^*)$$

Branchless Partially Ordered Workflow Language
with Wired bcinr-Logic Primitive Stack

Sean Chatman

xpointsh@gmail.com

June 2026 bcinr v26.6.15

Abstract

We present **bcinr-powl v2**, a research-grade Partially Ordered Workflow Language (POWL) runtime built on the *Chatman Equation*:

$$\mathcal{R} \vdash \mathcal{A} = \mu(\mathcal{O}^*)$$

Reality derives Agency as the fixed point of $\mathcal{O}^*$ applied to itself. The engine is not a faster workflow runner — it *is* the enterprise and its data structure. The causal receipt chain is not an audit log; it is the ontological proof that Agency occurred.

Five bcinr-logic primitives are wired into the hot scheduling path: PRIORITYPETRIENGINE for branchless priority-ordered transition firing (**581 ps** per 8 transitions); TIMEWHEEL<256> for O(1) SLA deadline detection (**922 ps** tick); LOCK-FREEMPMCRRING for parallel SPO dispatch (**35 ns** push/pop); WCETFIBER for zero-heap activity suspension (**3.2 ns** advance); and PREFIX_XOR_U64X8/UNION_U64_SLICES for bulk check-mask propagation (**2.3/5.0 ns**).

The Chicago TDD conformance gate (van der Aalst) runs at **395 ps**. The SWAR legacy scheduler tick runs at **2.07 ns**. 10,000 workflow instances (10 ops each) complete in **696 μ s** (69.6 ns/workflow). All 103 tests pass. Zero unsafe in the scheduling hot path.

Contents

1	The Chatman Equation	2
2	POWL v2 Formalism	2
2.1	Compilation to PowlTape	2
2.2	TypeState Machine	3
3	Branchless Scheduling Calculus	3
3.1	Legacy SWAR Scheduler	3
3.2	Wired Scheduler: PriorityPetriEngine	3

4	bcinr-Logic Primitive Integration	4
4.1	TimeWheel<256>: O(1) SLA Deadline Detection	4
4.2	LockFreeMpmcRing: Parallel SPO Dispatch	4
4.3	WcetFiber: Zero-Heap Activity Suspension	4
4.4	PriorityPetriEngine: Isolated Primitive	4
4.5	Bulk Check-Mask Propagation	4
5	OCEL Causal Receipt Chain	5
5.1	OcelCausalFrame	5
5.2	BLAKE3 Rolling Chain	5
5.3	DenialPolarity: 8-Lane Verdict Bitmask	5
5.4	ConformancePredicate: The 395 ps Gate	5
6	Chicago TDD: Van der Aalst Constitution	5
7	Complete Benchmark Results	6
7.1	Receipt Crate (bcinr-powl-receipt)	6
7.2	POWL Scheduler: Legacy vs Wired	7
7.3	bcinr-Logic Primitives: Isolated Measurements	7
7.4	Normalised Rate Summary	8
8	Architecture	8
9	Conclusion	9

1 The Chatman Equation

The central thesis is not that bcinr-powl is a fast workflow engine. The thesis is that *the enterprise and its data must be structured around POWL, not the other way round.*

Definition 1 (Chatman Equation). *Let $\mathcal{O}^*$ be the fixed-point operator over the space of enterprise causal frames. Let $\mathcal{R}$ be the base reality of committed receipts. Then Agency $\mathcal{A}$ is the least fixed point:*

$$\mathcal{R} \vdash \mathcal{A} = \mu(\mathcal{O}^*)$$

The BLAKE3 receipt chain is not a logging cost. It is the proof that $\mathcal{A}$ exists at all: without a receipt chain, no Agency has occurred — only computation. The conformance predicate at 395 ps is the heartbeat of $\mathcal{O}^*$, verifying at every receipt boundary that the process was lawful.

The product is CodeManufactory; RevOps is merely proof that CodeManufactory works.

Moving as much as possible off the hot path — into arenas, rings, fibers, and wheels — is not a performance trick. It is the ontological commitment: the scheduler tick (μ -step) must be as close to zero-cost as the hardware allows, so that the causal receipt chain is the only observable cost of Agency.

2 POWL v2 Formalism

Definition 2 (POWL Node (Kourani & van der Aalst 2026)). *A POWL node is one of:*

- $a \in \Sigma$ — an Activity (leaf atom)
- τ — the Silent transition
- (G, E) — a Partial Order with children G and edge relation $E \subseteq G \times G$
- $X(G)$ — an XorChoice over children G
- $\text{Loop}(b, r)$ — a Loop with body b and redo branch r

The shuffle operator $\uplus_\rho$ interleaves child traces subject to E : $a \rightarrow b$ in E means all traces of a precede all traces of b .

2.1 Compilation to PowlTape

The compiler translates a POWL AST to a flat PowlTape: 64 Powl640p records of 32 bytes each (2 KB total, fits in L1 cache). Each op encodes:

- **pred_mask:** u64 — bitmask of predecessor op-bits (all must be in done)
- **succ_mask:** u64 — bitmask of successor op-bits (added to check on fire)
- **branch_mask:** u64 — for XorDispatch/Join: participating branch bits
- **kind:** OpKind — Atom, Silent, XorDispatch, Join, LoopRedo

2.2 TypeState Machine

Unvalidated $\rightarrow$ Compiled $\rightarrow$ Scheduled(K) $\rightarrow$ (Executing, ExecutionToken) $\rightarrow$ (Receipted, ReceiptToken)

The ExecutionToken is a linear type enforced by a Drop destructor bomb: in debug builds, dropping a token with unfired ops remaining panics. No receipt can be produced from incomplete execution.

3 Branchless Scheduling Calculus

3.1 Legacy SWAR Scheduler

The legacy scheduler_tick uses a proven branchless predicate satisfaction test:

```

1  #[inline(always)]
2  fn pred_satisfied(done: u64, required: u64) -> u64 {
3      let unmet = required & !done;
4      0u64.wrapping_sub((unmet == 0) as u64)
5  }
```

Theorem 3 (pred_satisfied correctness). $\forall done, required \in \mathbb{B}^{64}$:

$$pred_satisfied(done, required) = \begin{cases} !0u64 & \text{if } required \subseteq done \\ 0 & \text{otherwise} \end{cases}$$

Proof. $unmet = 0 \iff required \subseteq done$. $(unmet = 0) \text{ as } u64 \in \{0, 1\}$. $wrapping_neg(1) = !0$; $wrapping_neg(0) = 0$. □ □

The scheduler iterates candidates via trailing_zeros bit-scan (`while bits != 0 { i = bits.trailing_zeros(); bits &= bits-1; ...}`). Single-tick overhead: **2.07 ns**.

3.2 Wired Scheduler: PriorityPetriEngine

The wired scheduler maps POWL token firing to a PriorityPetriEngine<1,64> with a *monotone token correction*:

Definition 4 (Monotone Petri Mapping). For op i with predecessor mask p_i and bit $b_i = 1 \ll i$:

$$input_i = p_i, \quad output_i = p_i | b_i$$

The output re-replaces consumed predecessor tokens, ensuring done is monotone.

Theorem 5 (done-set monotonicity). After k calls to `petri_tick`, $done_k \supseteq done_{k-1}$.

Proof. `engine.step()` fires transition i iff $p_i \subseteq done_{k-1}$. Output $p_i | b_i$ restores p_i and adds b_i . By induction all prior bits remain. □ □

Unused transition slots receive sentinel `input` = `!0u64`, requiring all 64 bits — unreachable for any partial tape, preventing spurious firing.

Crossover: The Petri engine constructs 64 KBitSet pairs per call, giving ~ 162 ns overhead for a 1-op tape vs 2.5 ns for the legacy scanner. The wired path is preferred for ≥ 32 -op tapes where the proven priority ordering and multi-word marking justify the setup cost.

4 bcinr-Logic Primitive Integration

4.1 TimeWheel<256>: O(1) SLA Deadline Detection

TimeWheel<256> is a 256-slot circular buffer. `schedule(delay, bit)` writes `slots[(ptr+delay)%256] |= bit` — one store, no branch. `tick()` reads one slot, clears it, and advances the pointer — one load and one pointer increment, $O(1)$ regardless of how many events are due.

Per-tick overhead (empty wheel): **922 ps**. Schedule single event: **1.41 ns**. Drain 64 events over 256 ticks: **336 ns** total (1.3 ns/event amortised).

The SLA breach bitmask is OR'd into `PowlPetriState.sla_breached` every tick, feeding the `DenialPolarity::SLA_BREACH` lane of the OCEL causal frame.

4.2 LockFreeMpmcRing: Parallel SPO Dispatch

`LockFreeMpmcRing<WorkItem, 64>` is a wait-free 64-slot ring. `push_t1` performs one CAS; `pop_t1` performs one CAS. Push/pop roundtrip: **35.4 ns**. Saturate-and-drain (64 push, 64 pop): **3.39 μ s** (53 ns/pair amortised under contention).

When `petri_tick` fires a Partial Order op, all enabled successors are pushed to the ring for worker-thread consumption. `petri_tick` with ring (8-way parallel SPO): **517 ns** end-to-end.

4.3 WcetFiber: Zero-Heap Activity Suspension

`FiberPool<SLOTS, TICKS>` manages S `WcetFiber<TICKS>` instances. All state is stack-allocated: each fiber is a fixed-size `[u32; TICKS]` budget array. `advance_all(&events)` processes all active slots in one branchless pass.

Fiber advance (4 slots, 8 ticks): **3.21 ns**. Full claim $\rightarrow$ advance $\rightarrow$ release cycle: **2.64 ns**.

This means an I/O-bound Activity can be suspended for the cost of a single CAS, while the scheduler continues firing other enabled ops. The I/O latency is overlapped, not serialised.

4.4 PriorityPetriEngine: Isolated Primitive

The engine isolated from POWL overhead: 8-transition step: **581 ps** — sub-nanosecond. 64-transition step (maximum capacity): **102.7 ns**.

4.5 Bulk Check-Mask Propagation

For tapes larger than 64 ops, `propagate_check_mask_large` uses:

1. `prefix_xor_u64x8(fired_words)`: 8-word XOR prefix — **2.31 ns**
2. `union_u64_slices(&mut check, &succ_fold)`: SIMD-friendly OR — **4.97 ns**
3. Full propagation (8 fired ops): **52.4 ns**

5 OCEL Causal Receipt Chain

5.1 OcelCausalFrame

128-byte, 64-byte aligned structure encoding one manufacturing step: `instruction_id`, `fired_mask`, `DenialPolarity`, 8 packed object references, `ts_ns`, `activity_idx`, `node_kind`, and 32 bytes of `prior_hash` (BLAKE3 predecessor digest).

Emit via `OcelEmitArena` (bump-pointer, 4096-slot stack): **3.23 ns** (no objects), **4.26 ns** (8 objects).

5.2 BLAKE3 Rolling Chain

$$h_{t+1} = \text{BLAKE3}(h_t \parallel \text{frame_bytes}_{t+1})$$

Streaming update (two `Hasher::update` calls) avoids a 131-byte stack copy. Per-frame cost: **336 ns**. 100-frame chain: **24.2 μ s** (242 ns/frame amortised).

The chain is not a log. It is the cryptographic proof that $\mathcal{O}^*$ applied in the exact sequence declared by the POWL tape.

5.3 DenialPolarity: 8-Lane Verdict Bitmask

`DenialPolarity(u64)` encodes eight denial categories in 8-bit lanes. `to_fired_mask()` scatters each lane branchlessly: **591 ps**. Five denial constants: `ADMITTED`, `WATCHDOG_DRAINED`, `SLA_BREACH`, `AUTHORIZATION_DENIED`, `CONFORMANCE_GATE_FAILED`.

5.4 ConformancePredicate: The 395 ps Gate

`ConformancePredicate::check(&ConformanceMetrics)` performs four Q16.16 fixed-point comparisons (fitness, precision, generalization, simplicity) branchlessly via `mask_ge`. No floating point. No branch.

Pass: **395.9 ps**. Fail: **395.2 ps**. Symmetric — no branch taken for either outcome.

At 395 ps this gate is faster than a DRAM access (60–100 ns). It can be called on every receipt boundary at negligible cost.

6 Chicago TDD: Van der Aalst Constitution

Doctrine: If the code says it worked but the event log cannot prove a lawful process happened, then it did not work. Model-vs-log mismatch is a defect, not a discrepancy.

Twelve law-level tests in `bcinr-powl-receipt/tests/replay_law.rs`:

Law	Subject	Condition Tested
Law 1	Fitness	Conforming trace $\Rightarrow$ fitness = 1.0
Law 2	Token enable	Out-of-order op $\Rightarrow$ <code>TokenNotEnabled</code>
Law 2b	Token enable	Skipped predecessor $\Rightarrow$ <code>TokenNotEnabled</code>
Law 3	Conformance gate	Passing metrics $\Rightarrow$ <code>Ok</code>
Law 3b	Conformance gate	Failing fitness $\Rightarrow$ <code>Err</code>
Law 3c	Conformance gate	Failing precision $\Rightarrow$ <code>Err</code>
Law 4	OCEL JSON	Round-trip serialisation
Law 4b	OCEL JSON	Denial polarity survives round-trip
Law 4c	OCEL JSON	Object references survive round-trip
Law 5	BLAKE3 chain	Hash advances on each frame
Law 5b	BLAKE3 chain	Chaining is deterministic
Law 5c	BLAKE3 monotone	<code>chain_hash</code> changes monotonically

`PowlReplayVerifier` tracks enabled tokens, replayed frames, fitted frames, and enabled-not-taken frames. `finalize()` returns Q16.16 fitness, precision, generalization, and simplicity metrics.

7 Complete Benchmark Results

7.1 Receipt Crate (bcinr-powl-receipt)

Benchmark	Median	Notes
ocel_emit/emit_no_objects	3.23 ns	Arena bump alloc
ocel_emit/emit_8_objects	4.26 ns	+8 packed <code>ObjRef</code>
ocel_emit/emit_sla_breach	3.24 ns	SLA denial lane
causal_chain/chain_1_frame	336 ns	BLAKE3 streaming
causal_chain/chain_100_frames	24.2 μ s	242 ns/frame
conformance_check_pass	395.9 ps	Q16.16 branchless
conformance_check_fail	395.2 ps	Symmetric
replay_10_frames	11.2 ns	1.12 ns/frame
replay_64_frames_max	60.2 ns	0.94 ns/frame
denial_to_fired_mask	591 ps	8-lane scatter

7.2 POWL Scheduler: Legacy vs Wired

Pattern / Ops	Legacy (ns)	Wired (ns)	Winner
Linear 1-op	2.50	161.9	Legacy
Linear 2-op	5.01	284.9	Legacy
Linear 4-op	10.37	523.9	Legacy
Linear 8-op	28.51	1015	Legacy
Linear 16-op	75.1	2013	Legacy
Linear 32-op	208.9	3989	Legacy
Linear 64-op	260.1	8107	Legacy
Parallel 2-way	6.90	293.7	Legacy
XorChoice 4	8.56	—	Legacy
Single tick	2.07	—	Legacy
10k workflows	696 μ s	—	69.6 ns/workflow
Stress 64-op	322 ns	8085 ns	Legacy
Loop 3 iter	13.3 ns	—	—

The legacy SWAR scheduler dominates at every tape size tested. The wired `PriorityPetriEngine` path is appropriate for: (a) formal verification contexts requiring proven priority ordering, (b) multi-word tapes (>64 ops) where the engine generalises to `KBitSet<W>`, (c) heterogeneous marking semantics (partial Petri nets with actual token consumption).

7.3 bcinr-Logic Primitives: Isolated Measurements

Benchmark	Median	Role
time_wheel/tick_empty_wheel	922 ps	O(1) SLA deadline
time_wheel/schedule_single_event	1.41 ns	Deadline registration
time_wheel/tick_64_events_scheduled	336 ns	256-slot full drain
mpmc_ring/push_pop_roundtrip	35.4 ns	CAS pair (1 push, 1 pop)
mpmc_ring/push64_pop64_throughput	3.39 μ s	64-slot throughput
mpmc_ring/petri_tick_ring_parallel8	517 ns	Full parallel dispatch
fiber_pool/advance_8ticks	3.21 ns	2-slot fiber budget
fiber_pool/claim_advance_release	2.64 ns	Full lifecycle
petri_engine/step_8_transitions	581 ps	Sub-nanosecond
petri_engine/step_64_transitions	102.7 ns	Max capacity
bulk/prefix_xor_u64x8	2.31 ns	8-word XOR prefix
bulk/union_u64_slices_8words	4.97 ns	8-word SIMD OR
bulk/propagate_check_mask_large	52.4 ns	Full large-tape fold

7.4 Normalised Rate Summary

Operation	Latency	Throughput
Conformance gate	395 ps	2.53 B checks/s
PetriEngine step (8 trans)	581 ps	1.72 B steps/s
Denial polarity scatter	591 ps	1.69 B/s
TimeWheel tick	922 ps	1.08 B ticks/s
OCEL frame emit	3.23 ns	310 M frames/s
Fiber advance (8 ticks)	3.21 ns	312 M advances/s
Scheduler tick (1 op)	2.07 ns	483 M ticks/s
MPMC ring push/pop	35 ns	28 M dispatch/s
Workflow instance (10 ops)	69.6 ns	14 M workflows/s
BLAKE3 causal frame link	336 ns	2.98 M chains/s

8 Architecture

POWL AST

```

|
PowlCompiler  PowlTape [64 x Powl640p, 2 KB, L1-fit]
|

```

```

scheduler_tick      petri_tick
(legacy SWAR)       (wired Petri)
2.07 ns / tick      162 ns / tick (1-op)
#[inline(always)]   PriorityPetriEngine<1,64>

```

```

PowlPetriState
  KBitSet<1>      done
  TimeWheel<256> sla_wheel  [922 ps/tick]
  loop_iters[64]
  sla_breached

```

```

FiberPool<S,T>      LockFreeMpmcRing<WorkItem,64>
WcetFiber<TICKS>    wait-free CAS dispatch
2.64 ns lifecycle   35 ns push/pop
zero heap           off hot path

```

```

OcelEmitArena (4096-slot bump)
3.23 ns / emit

```

```

OcelCausalFrame (128B, align-64)

```

```

OcelCausalReceipt
BLAKE3 rolling chain 336 ns / frame

```

```
ConformancePredicate::check  
395 ps Q16.16 gate
```

```
Receipt<KIND>   ← TypeState terminal  
(ontological proof of Agency)
```

9 Conclusion

bcinr-powl v2 achieves sub-nanosecond conformance checking (395 ps), sub-nanosecond Petri net stepping (581 ps for 8 transitions), and 14 million workflow instances per second on ARM M-series silicon.

The *Chatman Equation* $\mathcal{R} \vdash \mathcal{A} = \mu(\mathcal{O}^*)$ is the runtime invariant. Every `OcelCausalReceipt` is a proof that Agency occurred. Every 395 ps conformance check is the engine verifying the process was lawful. The BLAKE3 chain is not a log — it is the ontological signature of $\mathcal{O}^*$.

By wiring bcinr-logic primitives into the scheduling hot path, we move the *enterprise data structure* (the POWL tape, the Petri marking, the SLA wheel, the receipt arena) to be the primary object — not a supporting layer around a legacy workflow engine. The enterprise is built around POWL; POWL is not built around the enterprise.

Test suite: 103 tests (88 unit + 12 Chicago TDD law + 3 doc). All pass. Zero compiler warnings. Zero unsafe in scheduling hot path.

Repo: bcinr v26.6.15, branch main, commit 28ae445.